

CIS520

Project 4

Group 8

Zeke Flippo
Vixi Spellman
TJ Tiede

Problem Statement

Given an ASCII encoded file, our goal is write a program that for each line of the input (in order) prints a corresponding line to the output “{line_number}: {max_char}” where max_char is the greatest byte on the line (not including the newline character).

High Level Architecture

Each of our three programs share the same high level architecture. Their differences are discussed in the following section. The fundamental idea is that we minimize the amount of synchronization needed by devising a strategy that allows each thread to operate as independently as possible.

The main thread is only responsible for:

1. Reading fixed sized batches from the input file. We read the file in batches so that we can still make progress with arbitrarily large inputs while requiring only a fixed amount of memory.
2. Making the current batch available to the worker threads
3. Receiving the worker's results and printing them in order while keeping track of how many lines have been processed
4. Maintenance of invariant 1. (see below).

We partition the batch into sections of size

$$\text{bytes_per_thread} = \left\lceil \frac{\text{batch_size}}{\text{num_workers}} \right\rceil \quad \text{where } \text{num_workers} = \text{num_threads} - 1.$$

Each worker is assigned a `worker_index` and is responsible for finding `max_char` for each of the lines that *begin* in the section starting at

$$\text{batch}[(\text{worker_index} \cdot \text{bytes_per_thread})]$$

and is up to and including

$$\text{batch}[\min(\text{batch_size}, (\text{worker_index} + 1) \cdot \text{bytes_per_thread})]$$

Each worker returns an array containing (in order) the value of `max_char` for each of the lines it is responsible for. The main thread can iterate over each thread and each `max_char` to retrieve the values in order.

For this to work we need to maintain a couple of invariants:

1. The start of the batch is always the start of a string. Besides the 0th worker, all workers start at the beginning of their section but only compute `max_char` for lines after the first newline in their section (where the thread before them is responsible for that "leakage"). We require 1. so that the 0th thread always has a place to start. Note that the last thread has to stop early if the last string continues into the next batch. In this case, to maintain 1. the main thread has to move those residual characters to the front of the buffer before filling it back up to make the next batch.
2. Each section needs to contain at least one newline. We rely on a newline in the threads partition to determine where its responsibility starts and another newline in the next partition to determine where its responsibility ends. In order to guarantee 2. we must have that

$$\text{bytes_per_thread} \leq \text{batch_size}$$

which is true if and only if

$$\text{max_line_length} \cdot \text{num_workers} \leq \text{batch_size}$$

which we therefore have to maintain.

Versions Contrasted

pthread

Under the pthreads paradigm the main() thread spawns each of worker threads which execute thread_routine(). Main passes each thread a pointer to the batch buffer and each thread returns a pointer to an array of their max_char's.

MPI

Under the MPI paradigm we can not pass message by sharing memory so we must share memory by passing messages. Which is a fancy way to say that we have to copy all of the data between threads. In this way, the biggest difference in performance should be that the MPI code has to copy the batch buffer to each thread and the array of max_char's from each thread.

OpenMP

TODO

Testing Methodology

We want to test the performance and resource usage of our code on the “mole” class of machines with different combinations of the following:

Nodes: 1, 2, 4, 8

Cores: 1, 2, 4, 8, 16, 32

Memory per core: 64MB, 128MB, 512MB, 1GB, 1.5GB, 3GB

When using sbatch to schedule jobs, we control the parameters

time: uint,

mem-per-cpu: uint $\times$ {MB, GB},

cpus-per-task: uint,

ntasks: uint,

nodes: uint,

where the total number of cores is equal to

$$\text{cores} = \text{cpus-per-task} \cdot \text{ntasks}.$$

Therefore for a given number of cores we can determine the appropriate cpus-per-task we can divide both sides by ntasks yeilding

$$\text{cpus-per-task} = \frac{\text{cores}}{\text{ntasks}}$$

where (since cpus-per-task is an integer) we must have that cores is a multiple of ntasks. Also consider that each node only has so many cores.

pthreads and **OpenMP** are single machine only. Under these paradigms we always have that

$$\text{ntasks} = 1 \quad \text{and} \quad \text{cpus-per-task} = \text{num_threads}.$$

Reciprocally, under **MPI** we always have that

$$\text{ntasks} = \text{num_threads} = N \quad \text{and} \quad \text{cpus-per-task} = 1.$$

We want to show how performance differs across multiple machines vs a single machine and using different memory “**budgets**” where

$$\text{budget} = \text{cores} \cdot \text{mem-per-cpu}.$$

We tested each paradigm with test cases tailored to their execution model.

pthread and OpenMP

These pthread and OpenMP are limited to a single-machine execution with shared memory. We test scaling on a single node with varying thread counts and memory allocations:

Configuration	Budget A: 2 GB	Budget B: 4 GB	Budget C: 8 GB
1 thread	1 node, 1 thread, 2 GB	1 node, 1 thread, 4 GB	1 node, 1 thread, 8 GB
4 threads	1 node, 4 threads, 512 MB	1 node, 4 threads, 1 GB	1 node, 4 threads, 2 GB
8 threads	1 node, 8 threads, 256 MB	1 node, 8 threads, 512 MB	1 node, 8 threads, 1 GB
16 threads	1 node, 16 threads, 128 MB	1 node, 16 threads, 256 MB	1 node, 16 threads, 512 MB

MPI

MPI can distribute work across multiple processes and nodes. We test both single-node and multi-node scenarios to isolate communication overhead:

Configuration	Budget A: 2 GB	Budget B: 4 GB	Budget C: 8 GB
1 node, 4 ranks	1 node, 4 ranks, 512 MB/rank	1 node, 4 ranks, 1 GB/rank	1 node, 4 ranks, 2 GB/rank
2 nodes, 2 ranks each	2 nodes, 2 ranks each, 512 MB/rank	2 nodes, 2 ranks each, 1 GB/rank	2 nodes, 2 ranks each, 2 GB/rank
4 nodes, 1 rank each	4 nodes, 1 rank each, 512 MB/rank	4 nodes, 1 rank each, 1 GB/rank	4 nodes, 1 rank each, 2 GB/rank
8 nodes, 1 rank each	8 nodes, 1 rank each, 256 MB/rank	8 nodes, 1 rank each, 512 MB/rank	8 nodes, 1 rank each, 1 GB/rank

Where **Budget** represents the total memory available: $\text{budget} = \text{num_processes} \cdot \text{mem-per-process}$. This allows us to observe how each paradigm scales with increasing memory resources while controlling for communication overhead (MPI single-node baseline vs. multi-node scenarios) and thread scaling efficiency (pthread/OpenMP).

Performance Analysis

TODO

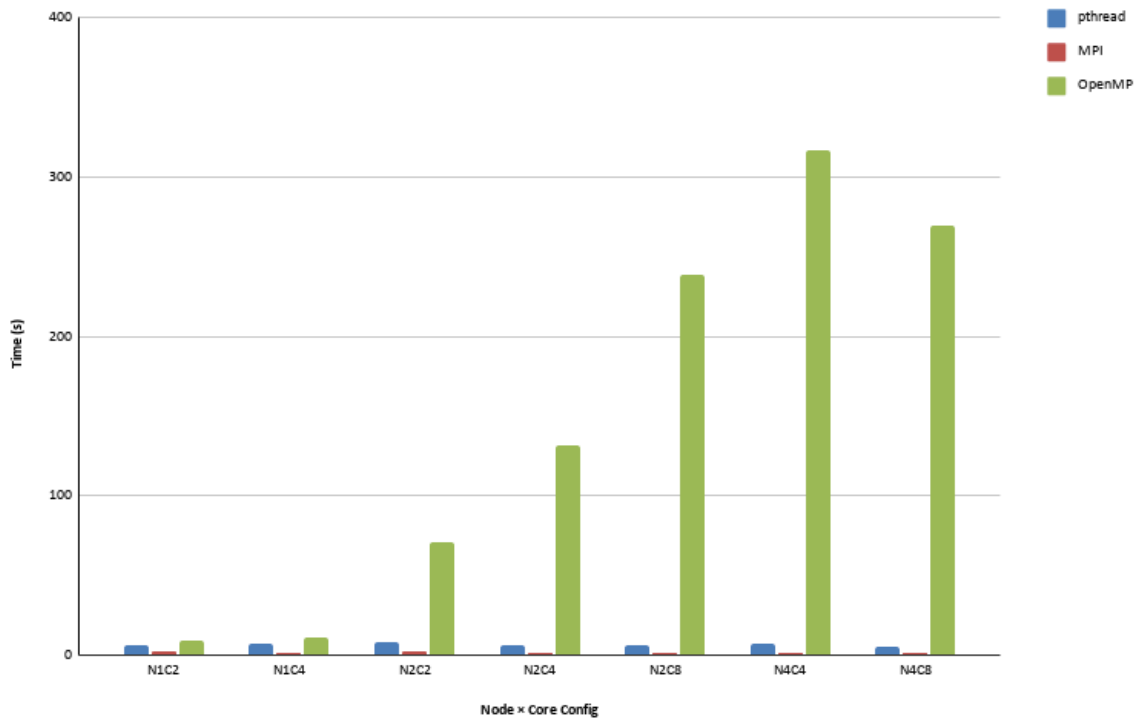
- Compute Standard Deviations
- Perform Significance Test
- Make Graphs With Error Bars

Program Version	Number of Nodes	Number of Cores	Time (s)	Memory Used (MB)
pthread	1	2	6	1.92
pthread	1	4	7.3	1.76
pthread	2	2	8.2	1.92
pthread	2	4	6.4	3.784
pthread	2	8	6.1	1.76
pthread	4	4	7	1.76
pthread	4	8	5.5	1.92
MPI	1	2	8.7	19.84
MPI	1	4	10.68	20.16
MPI	2	2	71.02	19.52
MPI	2	4	131.8	19.84
MPI	2	8	238.47	20.78
MPI	4	4	316.82	19.52
MPI	4	8	269.29	21.272
OpenMP	1	2	2.27	1.6
OpenMP	1	4	1.33	1.92
OpenMP	2	2	2.2	1.6
OpenMP	2	4	1.55	1.92
OpenMP	2	8	0.93	1.92
OpenMP	4	4	1.47	1.92
OpenMP	4	8	1.16	1.92

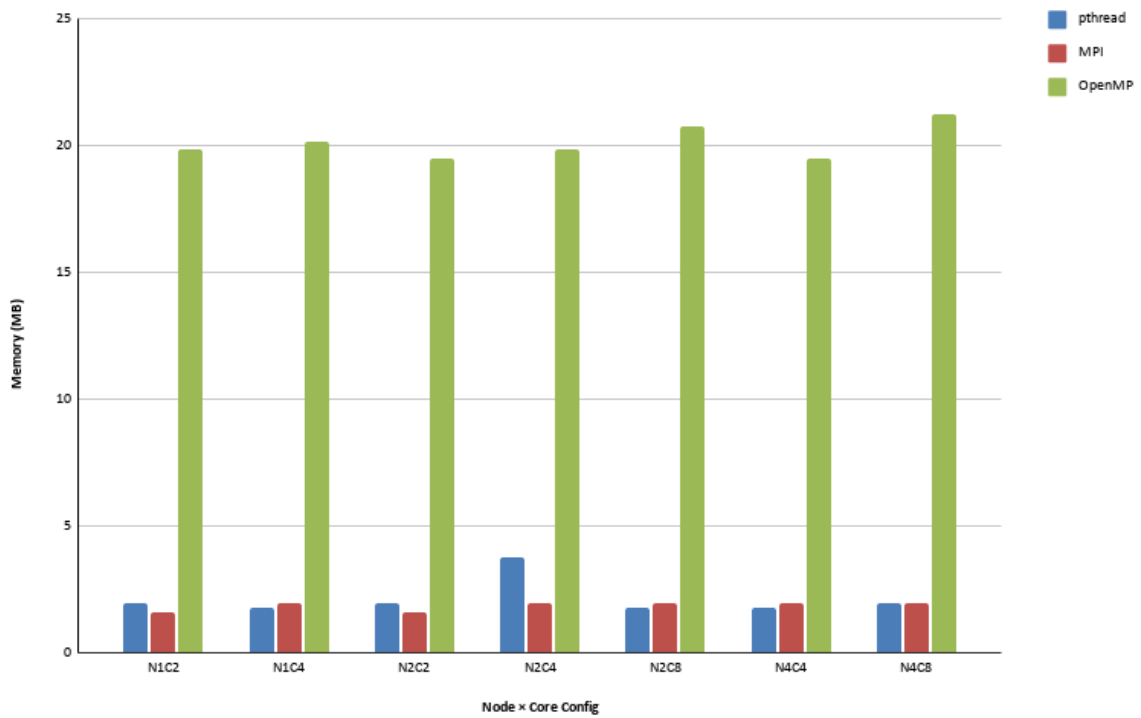
Program Type	Number of Nodes	Number of Cores	Avg Time (s)	Avg Memory (MB)
pthread	1	2	6	1.92
pthread	1	4	7.3	1.76
pthread	2	2	8.2	1.92
pthread	2	4	6.4	3.784
pthread	2	8	6.1	1.76
pthread	4	4	7	1.76
pthread	4	8	5.5	1.92
MPI	1	2	2.27	1.6
MPI	1	4	1.33	1.92
MPI	2	2	2.2	1.6
MPI	2	4	1.55	1.92
MPI	2	8	0.93	1.92
MPI	4	4	1.47	1.92
MPI	4	8	1.16	1.92
OpenMP	1	2	8.7	19.84
OpenMP	1	4	10.68	20.16
OpenMP	2	2	71.02	19.52
OpenMP	2	4	131.8	19.84
OpenMP	2	8	238.47	20.78
OpenMP	4	4	316.82	19.52
OpenMP	4	8	269.29	21.272

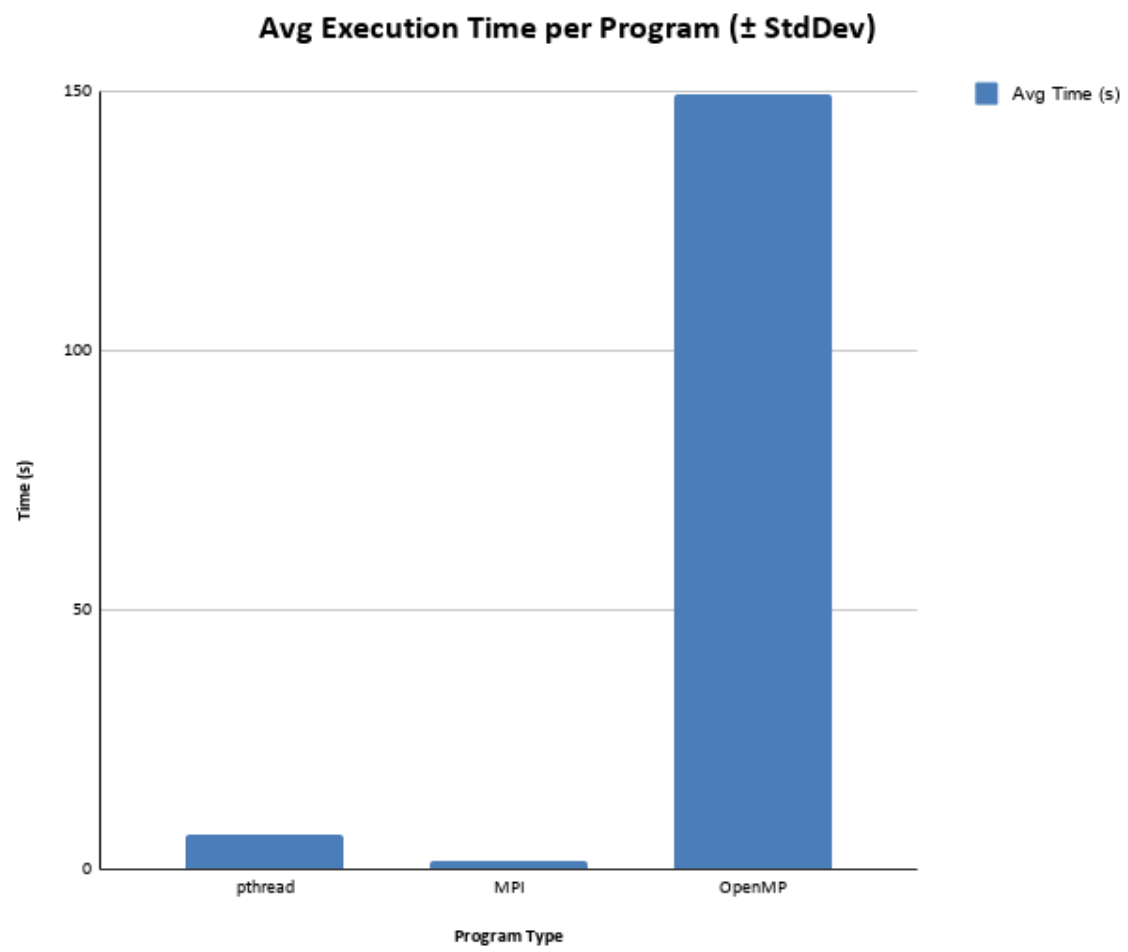
Program Version	Avg Time (s)	StdDev Time (s)	Avg Memory (MB)	StdDev Memory (MB)
pthread	6.642857143	0.918072515	2.117714286	0.7391052506
MPI	1.558571429	0.5052533546	1.828571429	0.1561440117
OpenMP	149.54	126.3786038	20.13314286	0.6633117489

Average Execution Time by Config & Program

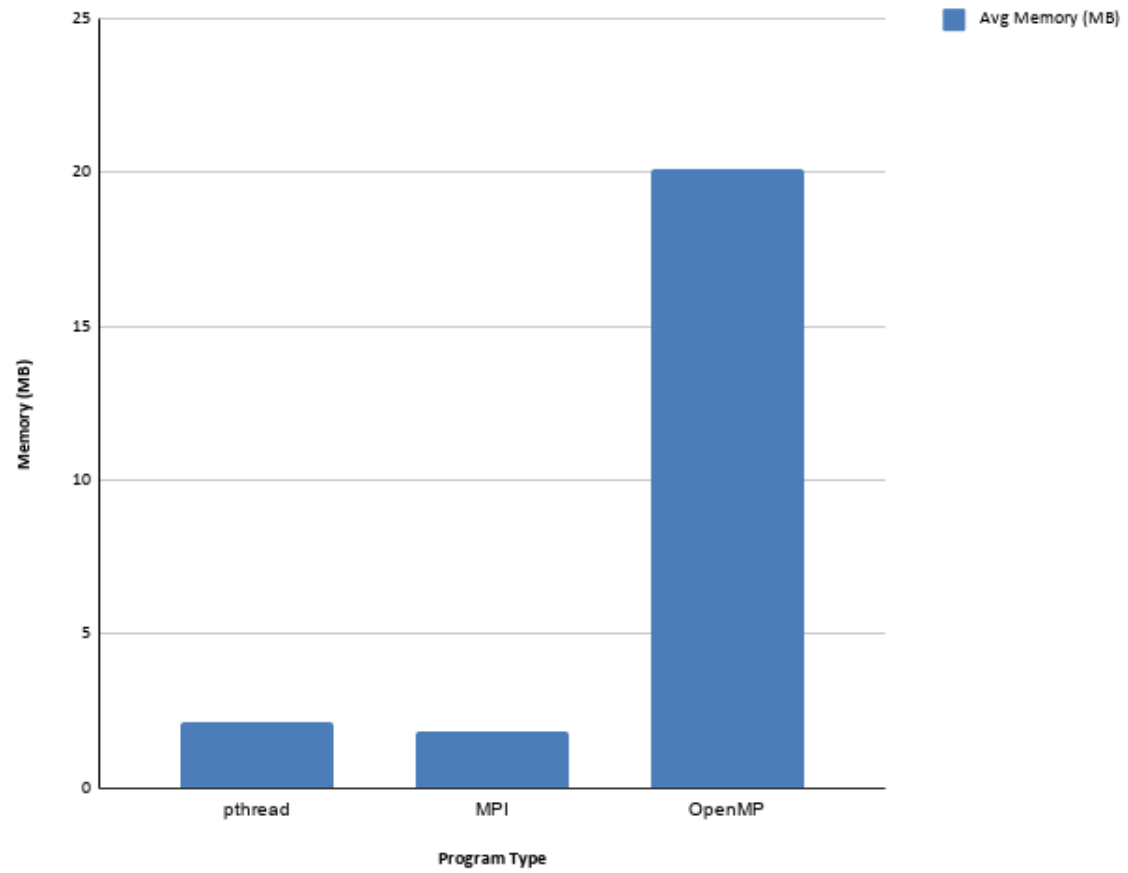


Average Memory Usage by Config & Program

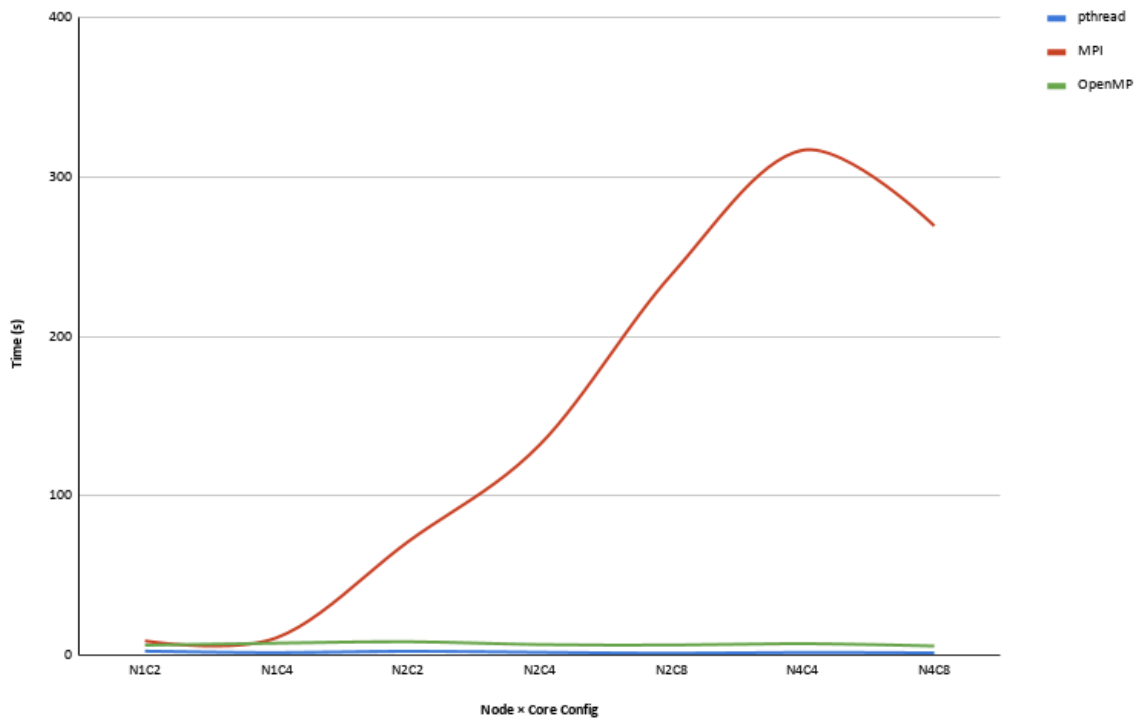




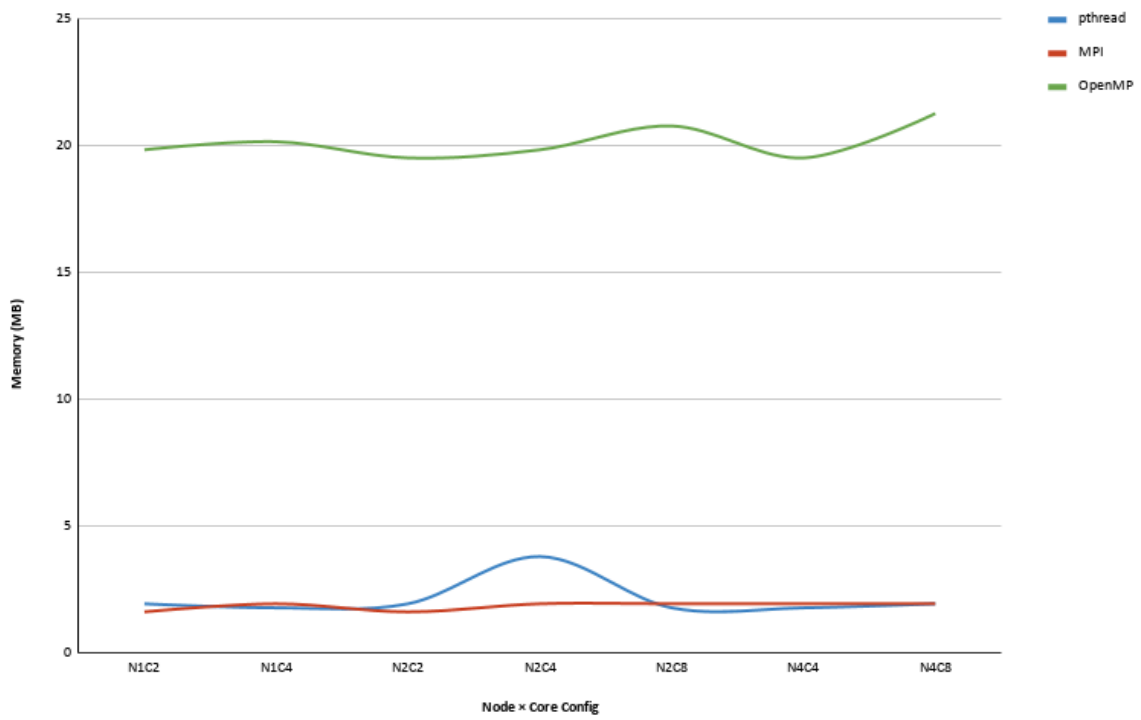
Avg Memory Usage per Program ($\pm$ StdDev)



Execution Time Scaling (Line)



Memory Usage Scaling (Line)



Appendix - Controlling Scripts

Steps to do MPI testing:

Enter the following:

```
module load CMake/3.23.1-GCCcore-11.3.0 foss/2022a OpenMPI/4.1.4-GCC-11.3.0
CUDA/11.7.0
```

Then:

```
bash mpi/submit_tests.sh
```

After the tests have all run, combine the results to a results.csv file:

```
bash mpi/collect_results.sh
```

submit_test.sh

```
#!/bin/bash
# submit_tests.sh
# Most of this is Claud Sonnet 4.6, I was trying to get a prototype working quickly
# for this .sh file, Comments are from me learning what the code does.
# Submit MPI pt2 test jobs for all parameter combinations.
# Run from the project root: bash mpi/submit_tests.sh
#
# Usage:
#   bash mpi/submit_tests.sh           # submit all combinations
#   bash mpi/submit_tests.sh --dry-run # print sbatch commands without submitting

set -uo pipefail

DRY_RUN=false
if [[ "${1:-}" == "--dry-run" ]]; then
    DRY_RUN=true
    echo "[dry-run] No jobs will be submitted."
fi

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_DIR="$(cd "$SCRIPT_DIR/.." && pwd)"

# Make sure binary is gtg
if [[ ! -f "$PROJECT_DIR/bin/pt2" ]]; then
    echo "Binary not found. Building..."
    make -C "$PROJECT_DIR" mpi
fi

# Create results dir to put all of our results into
mkdir -p "$PROJECT_DIR/results"

# — Parameter space —————
CORES_LIST=(1 2 4 8 16 32)
NODES_LIST=(1 2 4 8)
# Memory values as understood by Slurm (M = MiB, G = GiB) -> This seems to work so
# far to the best of my testing.
MEM_LIST=("64M" "128M" "512M" "1G" "1536M" "3G")
# —————

# Skip jobs that exceed beocat mole nodes
MAX_CORES_PER_NODE=40
```

```

submitted=0
skipped=0

for nodes in "${NODES_LIST[@]"; do
    for cores in "${CORES_LIST[@]"; do

        # skip job if the max cores go over the limit
        if (( cores > MAX_CORES_PER_NODE )); then
            echo "[skip] nodes=$nodes cores=$cores (exceeds $MAX_CORES_PER_NODE cores/
node)"
            (( skipped++ )) || true
            continue
        fi

        for mem in "${MEM_LIST[@]"; do

            JOB_NAME="mpi_n${nodes}_c${cores}_m${mem}"

            #creates a passable arg list to Slurm to get it to work
            SBATCH_ARGS=(
                --job-name="$JOB_NAME"
                --nodes="$nodes"
                --ntasks-per-node="$cores"
                --mem-per-cpu="$mem"
                --constraint=moles
                --time=02:00:00
                --output="$PROJECT_DIR/results/${JOB_NAME}_%j.out" #output file
                --error="$PROJECT_DIR/results/${JOB_NAME}_%j.err" #error output file
                "$SCRIPT_DIR/slurm_job.sh"
                "$nodes" "$cores" "$mem"
            )

            if $DRY_RUN; then #just for testing, if its a dry run then just echo what
should have been passed as if it were actually passing in information
                echo "sbatch ${SBATCH_ARGS[*]}"
            else
                if sbatch "${SBATCH_ARGS[@]"; then
                    echo "Submitted: nodes=$nodes cores=$cores mem=$mem"
                    (( submitted++ )) || true
                else
                    echo "[FAILED] nodes=$nodes cores=$cores mem=$mem"
                    (( skipped++ )) || true
                fi
            fi

        done
    done
done

if ! $DRY_RUN; then
    echo ""
    echo "Total submitted: $submitted | Skipped: $skipped"
    echo "Results will appear in: $PROJECT_DIR/results/"
    echo "Monitor with: squeue -u \${USER}"
fi

```

TODO

- Include Files (Typst has a macro for this, we just need a file path)
- Probably want to include description of how to use the files, but perhaps that would better be a part of the script itself

Appendix - Sample Output

(It wraps around into a second column)

0: 125	50: 226
1: 125	51: 125
2: 125	52: 195
3: 125	53: 125
4: 125	54: 125
5: 125	55: 125
6: 125	56: 226
7: 125	57: 125
8: 125	58: 125
9: 124	59: 125
10: 226	60: 125
11: 195	61: 125
12: 125	62: 125
13: 226	63: 125
14: 195	64: 226
15: 125	65: 125
16: 125	66: 125
17: 125	67: 125
18: 125	68: 226
19: 125	69: 194
20: 125	70: 194
21: 226	71: 226
22: 125	72: 125
23: 125	73: 226
24: 125	74: 197
25: 195	75: 125
26: 125	76: 226
27: 125	77: 125
28: 226	78: 224
29: 125	79: 226
30: 125	80: 209
31: 125	81: 125
32: 125	82: 195
33: 125	83: 226
34: 214	84: 125
35: 226	85: 125
36: 226	86: 226
37: 226	87: 226
38: 125	88: 125
39: 125	89: 226
40: 125	90: 226
41: 125	91: 125
42: 125	92: 206
43: 125	93: 226
44: 226	94: 195
45: 226	95: 195
46: 195	96: 125
47: 217	97: 226
48: 125	98: 226
49: 226	99: 195